Informe post-incidente — TP Unidad 6: mantenimiento, respaldo y recuperación

Food Store · Base de Datos II · Tecnicatura Universitaria en Programación (UTN) · PostgreSQL 17 sobre Windows · Simulacro del 2026-10-18 · Informe del 2026-10-21

1. Respaldo utilizado y simulacro

Se restauró el último respaldo, **foodstore_20261018_094932.dump**, creado el **2026-10-18 a las 09:49:32** (formato custom, 10.650.048 bytes, SHA-256 9FF281D9…B55FB; *pg_restore --list* correcto). Se generó a mano al probar *backup_foodstore.ps1*: no hay tarea programada. Se restauró en **foodstore_restore_test** (nunca sobre *foodstore*). Se hicieron dos corridas de *restore_test.ps1*: la primera encontró un error del script (ver punto 4) y la segunda, con el script corregido, es la **oficial**: cronómetro de 11:16:42 a 11:16:48, **tiempo total de restauración y validación: 6,19 s** (la primera tardó 5,51 s).

2. RPO y RTO

	Previsto (laboratorio)	Observado (corrida oficial)	Cómo se calculó
RPO (datos que se pueden perder)	hasta 24 h (respaldo diario)	**1,45 h** (01:27:10)	Inicio del simulacro (11:16:42) menos creación del respaldo (09:49:32)
RTO (tiempo para recuperar)	menos de 20 min	**6,19 s**	Cronómetro desde antes de crear la base hasta el fin de las validaciones

El RPO es sobre **datos**: mide qué tan viejo era el respaldo (en un incidente a esa hora se habrían perdido como máximo 1,45 h de cambios). El RTO es sobre **tiempo**: cuánto tarda volver a operar. Son independientes. Ambos cumplen lo previsto, con dos salvedades: los respaldos fueron manuales y el RTO es de una base de prueba (~10,6 MB) en el mismo equipo, no extrapolable a producción; no incluye un *ANALYZE* posterior.

3. Validaciones de tablas y restricciones lógicas

Control	Antes del respaldo	Restaurada	Coincide
categoria	6	6	Sí
usuario (la consigna dice *cliente*; en esta base se llama *usuario*)	20.006	20.006	Sí
producto	50.016	50.016	Sí
pedido	200.006	200.006	Sí
detalle_pedido	400.013	400.013	Sí
productos_invalidos (precio < 0 o stock < 0; debe ser 0)	0	0	Sí
detalles_invalidos (cantidad <= 0 o precio_unitario < 0; debe ser 0)	0	0	Sí

Todas las cantidades coinciden y los dos controles lógicos dan cero. La comparación contra los conteos previos (*conteos_antes.txt*) la hizo una persona: el script imprime los valores pero no los compara.

4. Problema real encontrado

restore_test.ps1, línea 152. En la primera corrida la base *foodstore_restore_test* no existía, *psql* no devolvió filas y la línea `$existeBase = (([string]$resultadoExiste).Trim() -eq "1")` falló con "No se puede llamar a un método en una expresión con valor NULL". El error no abortó el script ni quedó en el log (solo en la consola), y esa corrida terminó igual con RESULTADO: OK. Se corrigió con `("$resultadoExiste".Trim() -eq "1")`, se comprobó la sintaxis y se repitió el simulacro: la segunda corrida pasó por la rama que elimina la base existente, sin errores. También aparecieron un *ParserError* en *backup_foodstore.ps1* (línea 43, `$Servidor:$Puerto` sin llaves) y, en *maintenance.sql*, estadísticas con retraso que mostraban 0 tuplas muertas antes del *VACUUM* y 50.015 después; ambos se detectaron al ejecutar, no al leer.

5. Mejora concreta para la próxima ejecución

Respaldar también los roles con `pg_dumpall --globals-only --no-role-passwords` como un paso más de *backup_foodstore.ps1*. *pg_dump* no incluye roles ni permisos, y la restauración con *--no-owner --no-privileges* tampoco los trae: en el simulacro la base volvió con sus datos pero **sin dueños ni permisos**, y en un desastre real, sobre un servidor nuevo, habría que rearmar los roles a mano antes de dar acceso. Reduce el tiempo de recuperación real, que el RTO de 6,19 s no refleja. Es una **propuesta, no implementada ni probada**; las contraseñas no se incluyen y habría que reasignarlas.

6. Qué parte fue asistida con IA y cómo se verificó

Parte	Asistencia	Verificación antes de aceptarla
backup_foodstore.ps1	OpenCode generó el script completo desde *SPEC_U6.md* y *AGENTS.md*	Lista de ocho preguntas, ejecución manual del caso de éxito y del de error. La ejecución encontró un error de sintaxis que la lectura no vio
restore_test.ps1	OpenCode, con un prompt preparado con Claude	Lectura con la lista, análisis de sintaxis, prueba de rechazo con *-BaseDestino foodstore* (código 1) y dos ejecuciones reales; la primera mostró el error de la línea 152
health_check.sql	Claude (no OpenCode)	Revisado a mano (solo lectura, no cuenta la propia sesión) y ejecutado sin errores
maintenance.sql	OpenCode generó la primera versión; Claude ayudó a corregirla	Ocho preguntas, que no vieron el retraso de estadísticas; se detectó al leer la salida y se corrigió con pausas *pg_sleep*. Ejecución final: 0, 50.015 y 0 tuplas muertas

En ningún caso se aceptó código sin ejecutarlo: la revisión humana y la prueba real quedaron por encima de lo que propuso la herramienta. Evidencias en la carpeta *evidencias/* del repositorio.